

Relazione del progetto “BriscOOla”

Adam Paolo Razzino
Andrea Reggiani
Maisam Noumi
Riem Boukhama

23 giugno 2026

Indice

1	Analisi	3
1.1	Descrizione e requisiti	3
1.2	Modello del Dominio	4
1.2.1	Entità Principali	5
2	Design	7
2.1	Architettura	7
2.2	Design dettagliato	9
2.2.1	Andrea Reggiani	9
2.2.2	Riem Boukhama	13
2.2.3	Maisam Noumi	17
2.2.4	Adam Paolo Razzino	21
3	Sviluppo	25
3.1	Testing automatizzato	25
3.2	Note di sviluppo	26
3.2.1	Adam Paolo Razzino	26
3.2.2	Andrea Reggiani	27
3.2.3	Riem Boukhama	28
3.2.4	Maisam Noumi	28
4	Commenti finali	30
4.1	Autovalutazione e lavori futuri	30
4.1.1	Adam Paolo Razzino	30
4.1.2	Andrea Reggiani	31
4.1.3	Riem Boukhama	31
4.1.4	Maisam Noumi	32
A	Guida utente	33
A.1	Menu Principale	33
A.2	Fase di Gioco	34

A.3	Termine della partita	35
A.4	Visualizzazione Leaderboard	36
B	Esercitazioni di laboratorio	37
B.1	adampaolo.razzino@studio.unibo.it	37

Capitolo 1

Analisi

1.1 Descrizione e requisiti

Il Software BriscOOla è ispirato all'omonimo gioco di carte italiano. Il mazzo é costituito da 40 carte, distinte in 4 semi ordinati secondo una gerarchia di forza.

Ogni tipologia di seme possiede 5 carte denominate *figure* e tali carte sono le uniche ad avere un valore aggiunto di punti.

L'obiettivo è accumulare il maggior numero di punti possibile attraverso le *prese*, ovvero l'aggiudicazione delle carte durante i turni.

Prima dell'inizio della partita il mazzo viene mescolato e vengono distribuite 3 carte a ciascun giocatore.

Viene poi scoperta una scarta, il cui seme diventa la *Briscola*, cioè il seme dominante per l'intera partita. La carta viene poi messa in fondo al mazzo.

Ogni round si svolge in senso orario a partire dal vincitore del round precedente, o dal giocatore al primo round.

Ciascun giocatore, a turno, gioca una carta sul tavolo; il seme della prima carta giocata determina il *seme di riferimento* per il turno corrente.

Al termine del round, il giocatore che ha calato la carta più forte si aggiudica tutte le carte sul tavolo, incrementando il proprio punteggio.

La partita termina quando il mazzo è esaurito e tutti hanno giocato le carte in mano. Vince chi ha totalizzato più punti.

Requisiti funzionali

- **Menù principale:** Il giocatore deve visualizzare una schermata iniziale con possibilità di selezionare leaderboard e numero di giocatori.

- **Informazioni di Gioco:** Durante la partita l'utente avrà modo di controllare la gerarchia delle carte e il seme di briscola.
- **CPU Avversaria:** Deve essere presente una CPU in grado di sfidare l'utente e cercare di impedirgli di eseguire le prese.

Requisiti non funzionali

- **Punteggio e Salvataggio della partita:** Il punteggio finale viene salvato su file in caso di vittoria e consultabile tramite una leaderboard che mostra i dieci migliori risultati registrati.
- **Difficoltà CPU:** Il giocatore può selezionare la difficoltà della CPU contro cui giocare (Facile, Normale e Difficile). A ogni livello è presente un moltiplicatore che incide sul punteggio finale.
- **Visualizzazione delle prese:** Durante la partita è visibile la pila delle carte vinte da ciascun giocatore, aggiornata al termine di ogni round.
- **Portabilità:** Il gioco dovrà risultare fluido e dinamico, e poiché è stato sviluppato in Java risulta eseguibile su qualsiasi sistema con JVM installata.

1.2 Modello del Dominio

Il gioco è composto da molteplici turni, nei quali i *giocatori* calano una *carta* dalla propria mano sul tavolo. Al termine di ogni turno viene determinata una *presa*, assegnata al giocatore che ha giocato la carta più forte.

Le interazioni fra carte sono governate da due semi dominanti: il *seme di Briscola*, scelto a inizio partita e valido per tutta la durata, e il *seme di riferimento*, determinato dalla prima carta giocata nel turno corrente e valido solo per quel turno.

All'inizio di ogni turno ciascun giocatore pesca una carta dal *mazzo* (il vincitore del turno precedente gioca per primo).

Questo ciclo si ripete fino all'esaurimento delle carte, dopodiché viene effettuato il conteggio dei punti per determinare il vincitore.

Ad ogni carta è associato un *punteggio* che ne riflette la forza, indipendentemente dal seme: Di seguito sono elencati i punteggi associati ai valori delle carte:

- **Asso:** Undici punti

- **Tre:** Dieci punti
- **Re:** Quattro punti
- **Cavallo:** Tre punti
- **Fante:** Due Punti
- **Due - Sette:** Zero Punti

1.2.1 Entità Principali

- **Mazzo:** Composto da 40 carte, suddivise in quattro semi (Spade, Coppe, Bastoni, Denari), ciascuno da 10 carte e con un totale di 30 punti disponibili.
- **Carta:** Definita da un *Seme* e da un *Valore*. Cinque carte per seme hanno un punteggio associato rilevante ai fini della vittoria.
- **Giocatore:** Ogni giocatore deve avere una *Mano* di massimo 3 carte e un *mazzetto* contenente le carte vinte.
- **Tavolo:** L'area di gioco in cui vengono calate le carte del turno corrente. Ospita inoltre le *pile* delle prese effettuate da ciascun giocatore e il mazzo residuo.
- **Turno:** Èntità che gestisce lo svolgimento di un singolo round, memorizza le carte giocate e coordina l'ordine di gioco tra i giocatori.

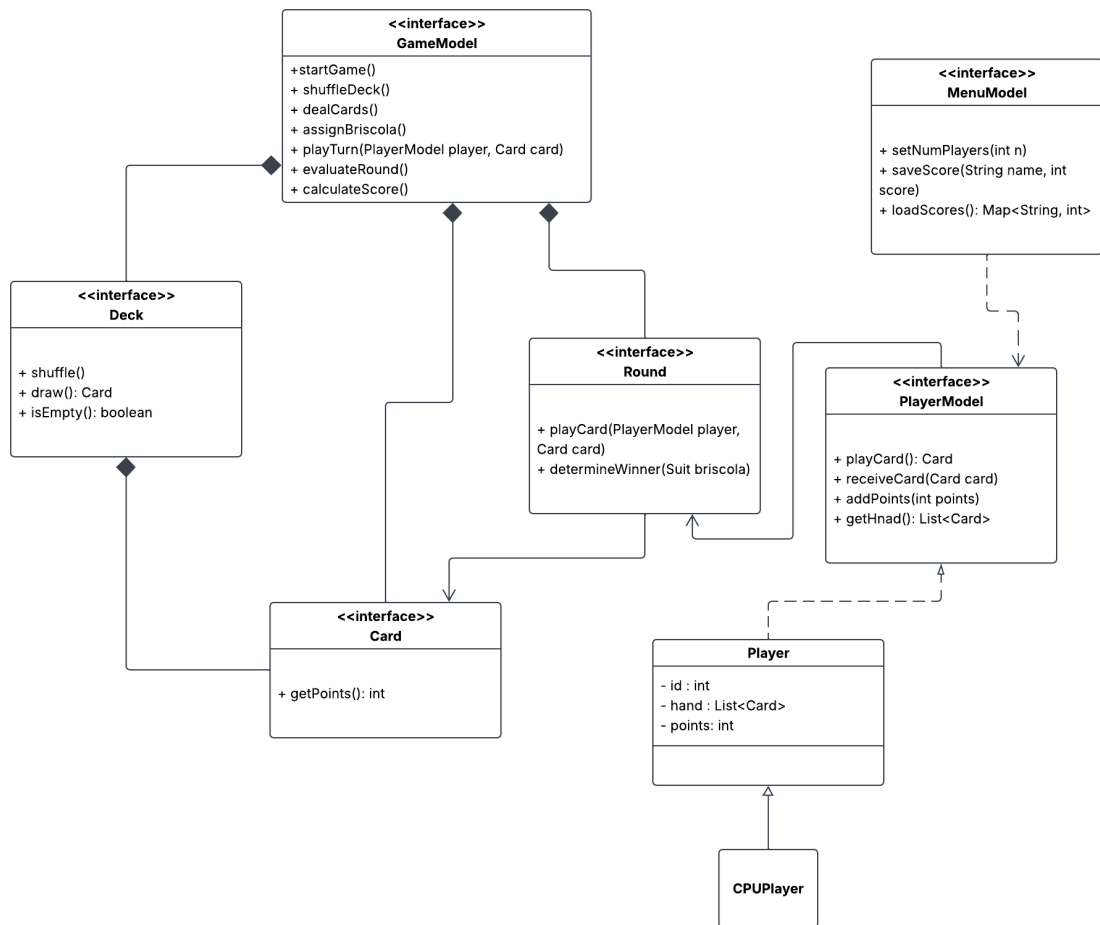


Figura 1.1: Schema UML dell'analisi del dominio, con rappresentate le entità principali ed i rapporti fra loro

Capitolo 2

Design

2.1 Architettura

L'architettura del gioco BriscOOla segue rigorosamente il pattern architetturale MVC, che garantisce una netta separazione dei ruoli e compiti tra Model, View e Controller.

Questa separazione garantisce una forte flessibilità, consentendo di rinnovare in maniera efficiente l'interfaccia grafica, senza interagire con le logiche principali dettate nel Model e nel Controller.

Il Model rappresenta il cervello dell'applicazione, gestisce lo stato della partita ed è strutturato attorno a entità isolate, come il mazzo di carte Deck, lo stato del turno e il calcolo dei punteggi. Non ha alcuna conoscenza della GUI e comunica con l'esterno solo tramite dati primitivi o immutabili.

La gestione del ciclo di vita dell'applicazione è suddivisa in due controller distinti:

- *GameController*: Coordinata lo scambio di informazioni tra il Model e la View durante la partita. Opera esclusivamente tramite le interfacce delle entità coinvolte, garantendo indipendenza dalla loro implementazione concreta e permettendo una eventuale sostituzione dell'interfaccia.
- *MenuController*: Definisce le firme pubbliche necessarie alla GUI per gestire la schermata iniziale, tra cui la lettura della classifica tramite il metodo `getLeaderboardData()`.

La *View* si interfaccia esclusivamente con i contratti astratti dei controller. L'invio dei dati di input (come il nome del giocatore e il livello di difficoltà) verso il `MenuControllerImpl` avviene attraverso l'uso di interfacce

funzionali (BiConsumer e Consumer). La *View* registra dei listener su eventi astratti, come *onGameStartListener*: al click dell'utente, invoca il metodo *accept()* delegando la responsabilità decisionale al *MenuController*.

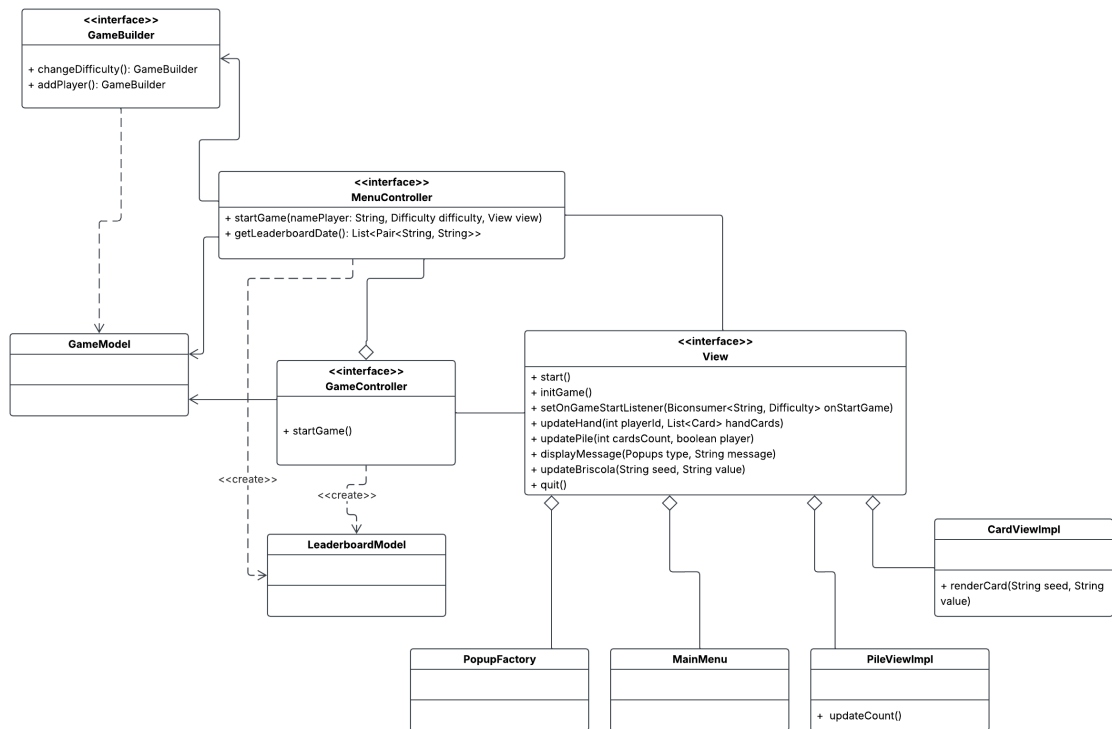


Figura 2.1: Diagramma dopo l'analisi del dominio

2.2 Design dettagliato

2.2.1 Andrea Reggiani

Generazione e Inizializzazione del Mazzo

Problema All'avvio di ogni partita di BriscOOla, il tavolo di gioco necessita di un set di 40 carte. Ogni carta deve essere definita in modo immutabile da un *Seme* e da un *Valore*, ai quali sono legati due valori, ossia i punti effettivi assegnati dalla carta a fine partita e la forza gerarchica della carta all'interno di una singola mano. La sfida consiste nel popolare il mazzo evitando che classi esterne possano alterare i dati delle carte durante il corso della partita e garantire un disaccoppiamento tra la logica di inizializzazione e i controllori che gestiscono l'andamento del gioco.

Soluzione Per rispondere a tale problematica, con un codice che risponda ai requisiti di robustezza manutenibilità, la soluzione prevede l'applicazione dello Strategy Pattern con un incapsulamento basato sulle interfacce, come mostrato nel diagramma UML (figura 2.2), con una separazione totale tra i contratti astratti e le loro implementazioni fisiche.

La struttura finale si divide in *Interfacce* e *Implementazioni*: Le Interfacce *Deck* e *Card* definiscono i comportamenti e contratti pubblici delle classi. I componenti esterni, come i Controller e View, interagiscono interamente ed esclusivamente con queste interfacce, senza vederne i dettagli di implementazione dei metodi, ma esclusivamente la loro dichiarazione dei metodi.

Le Implementazioni *DeckImpl* e *StandardCardImpl* forniscono la logica completa e le strutture interne, campi, appunto implementando la firma dei metodi dichiarati dalle loro corrispettive Interfacce, rimanendo non visibili alle altre classi dell'applicazione.

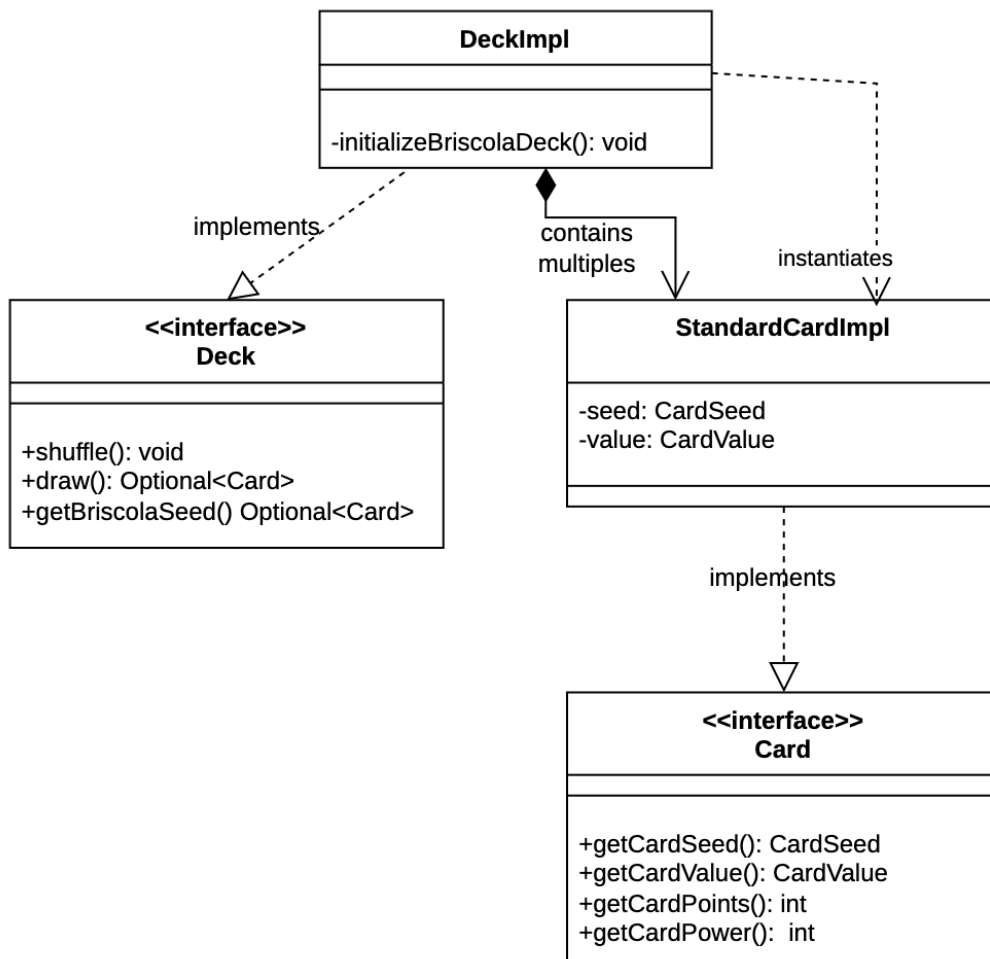


Figura 2.2: Rappresentazione UML del pattern Strategy

Gestione del Flusso d'Avvio e Configurazione dei Menu

Problema Nelle applicazioni sviluppate secondo l'architettura Model-View-Controller (MVC), un momento complicato risulta essere il passaggio tra la schermata del menu iniziale e l'arena di gioco vera. Se l'interfaccia grafica iniziale **StartScreen** fosse in grado di dialogare direttamente con i componenti del **Model**, si verificherebbe un accoppiamento rigido, che violerebbe il principio di separazione dei compiti.

La sfida risiede nel progettare un meccanismo che sia in grado di raccogliere e sanare i dati provenienti dalle componenti grafiche di input, che sia in grado di verificare che i dati inseriti siano nulli, che la difficoltà selezionata non sia inaccettabile e che in caso di errore il sistema sia in grado di fornire una risposta adeguata.

Soluzione La soluzione prevede l'introduzione di un controllore isolato, l'interfaccia **MenuController** e la sua rispettiva classe concreta implementativa **MenuControllerImpl**.

Il **MenuController** definisce le firme pubbliche necessarie alla GUI per notificare l'attivazione del metodo *startGame* o per estrarre in sola lettura i dati memorizzati delle classifiche storiche con *getLeaderboardDate*. La **GameViewImpl** dialoga esclusivamente con questo contratto astratto.

Il *MenuControllerImpl* fornisce l'implementazione della gestione. Non definisce in maniera diretta parametri che abbiano come valore le componenti grafiche, ma interagisce con essi esclusivamente come dipendenze locali all'interno del ciclo di vita dei metodi interessati. In una versione precedente ed iniziale si prevedeva una struttura semplificata dove il pannello di avvio **StartScreen** aveva il ruolo di creazione diretta del modello di gioco **GameModel**, la **View** era quindi autorizzata ad importare i pacchetti del modello.

Questo comportava un accoppiamento ovviamente contro il modello del MVC: la **View** dipende dal Model per crearlo, e il Model dipende dalla **View** per mostrare i dati.

In caso di cambiamenti nel Model, come il cambio della struttura della classe Card, l'intera interfaccia grafica andrebbe riscritta, una considerazione dovuta alla iniziale idea, nelle Funzionalità Opzionali, di gestire anche carte speciali, che differissero dalla **StandardCardImpl**.

Inoltre i dati inseriti dall'utente da tastiera, tramite i componenti di Swing, sono stringhe grezze e non controllate.

Permettendo alla componente grafica di avviare il dominio logico senza un controllo attivo espone il sistema a casi di accettazione di parametri e stati inconsistenti, come nomi utente vuoti e mancanti e/o errate configurazioni della difficoltà della CPU IA.

La soluzione raggiunta prevede che il trasferimento dei dati d'input dell'Utente non avvenga tramite una chiamata della **View** al **Controller**, ma mediante l'uso di interfacce funzionali standard di Java come i **Consumer**.

La **View**, nella sua implementazione finale, non conosce l'implementazione del **Controller**, e si limita a invocarne il metodo *accept()* sul *listener*, garantendo che i pannelli di **View** rimangano isolati.

La sanificazione e controllo dei dati in ingresso avviene per mezzo della classe **MenuControllerImpl**, che con il suo ruolo di **Controller**, che,

ricevendo i dati dalla **View** applica il metodo *trim* sul inserimento del nome del giocatore per poter riportare nella **leaderboard** (tabella dei risultati dei Player).

Una volta convalidati i dati, il controllore utilizza la classe **GameBuilderImpl** per configurare e generare l'istanza del modello **GameModel**, aggiungendo i giocatori richiesti e impostando la difficoltà dell'IA e invia alla vista l'ordine di inizializzare l'arena di gioco *view.initGame()*.

Infine istanzia il controllore di gioco principale, **GameControllerImpl**, passandogli il **Model** e la **View** appena configurati, per cedere il controllo del flusso alla logica della partita vera e propria.

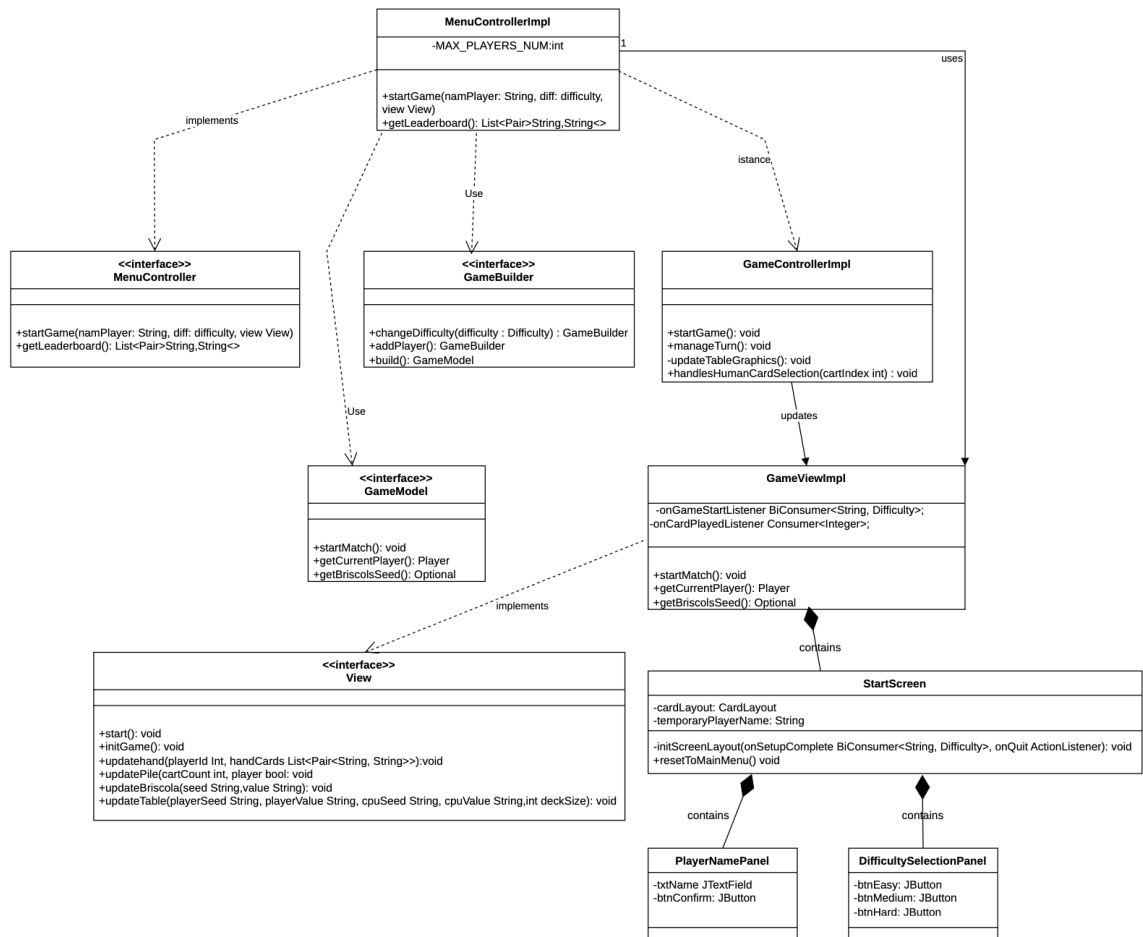


Figura 2.3: Rappresentazione UML delle interazioni e dipendenze del MenuController

2.2.2 Riem Boukhama

Disaccoppiamento tra logica di gioco ed entità giocatore

Problema Affidare la gestione del player al `GameModel` e al `GameController` renderebbe la logica centrale troppo complessa. Invece di incentrarsi sul flusso della partita, le classi di gestione del gioco si troverebbero a gestire i dettagli interni di ogni giocatore, complicando inutilmente il codice. Si creerebbe così un tight-coupling e le classi di logica finirebbero per dipendere strettamente dalle strutture dati interne dei giocatori. Di conseguenza, ogni modifica alla struttura giocatore costringerebbe a riscrivere la gestione di quest'ultimo nella partita, rendendo il sistema rigido. Il problema architetturale consiste quindi nel permettere un disaccoppiamento tra la logica della partita e le strutture dati dei singoli giocatori.

Soluzione Il problema è stato risolto progettando l'interfaccia `Player` e la classe `PlayerImpl` per esporre un'API che permette un'astrazione del giocatore e delle sue funzioni. La classe `PlayerImpl`, come da incapsulamento, non espone le sue strutture dati, ma fornisce metodi che riflettono le azioni reali del gioco: `receiveCard(Card)`, `playCard(RoundStateImpl state)`, `addtoPile(Card)`. La gestione degli array, la rimozione degli elementi e l'incremento del punteggio avvengono all'interno di questi metodi. Grazie a questa interfaccia, il `GameModel` e il `GameController` possono limitarsi a usare metodi semplici e completi, facilitando il flusso della partita.

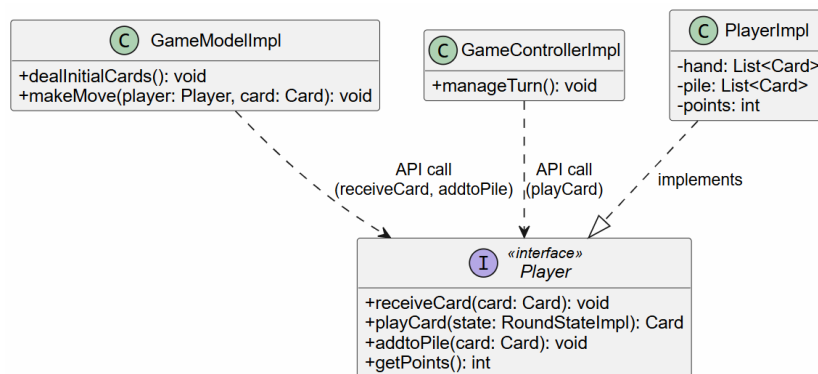


Figura 2.4: Rappresentazione UML dell'utilizzo di `Player` da parte di `GameModel` e `GameController`

Disposizione degli elementi nel tavolo

Problema Per offrire un'esperienza di gioco ottimale, l'ambiente della partita deve presentarsi all'utente in modo intuitivo e il più simile possibile al gioco della Briscola. Il giocatore si aspetta una disposizione classica di un tavolo da gioco: la propria mano vicino a sé (in basso), le carte coperte dell'avversario di fronte (in alto) e le carte giocate posizionate al centro del tavolo.

L'elemento di disturbo in questa simmetria è il mazzo di pescata con la relativa briscola scoperta. Essendo elementi che devono risiedere a sinistra, essi "spostano" il centro del tavolo verso destra. Se l'interfaccia non compensa questa asimmetria, l'area in cui vengono posate le carte giocate risulta disallineata rispetto ai giocatori. Il problema architettureale consiste quindi nel garantire al giocatore il posizionamento corretto delle zone del tavolo.

Soluzione Il problema è stato risolto sfruttando la componentizzazione e l'annidamento dei Layout Manager di Java Swing. L'intera finestra è gestita da un `BorderLayout` che divide lo spazio nelle aree del tavolo: nord per la CPU, sud per l'utente, ovest per il mazzo, centro per le carte giocate e visivamente estende nord e sud a destra per le carte vinte ad ogni round. La classe `GameViewImpl` delega a ciascuna macro-area il compito di organizzare i propri elementi tramite layout specifici (es. `FlowLayout` per allineare orizzontalmente in modo fluido le carte in mano).

Per posizionare le carte giocate al centro del tavolo è stato utilizzato un `FlowLayout`, che permette un inserimento fluido delle carte alla loro selezione. Questo layout, per sua natura, permette solo una distribuzione omogenea degli elementi ma non la loro posizione assoluta. Per ovviare al problema, il pannello centrale è stato inserito in un wrapper del tipo `GridBagLayout`, che permette di default di centrare il pannello. Infine, per restituire al giocatore la percezione di un tavolo centrato, è stato inserito un pannello non visibile nell'area Est, con una larghezza fissa calcolata per bilanciare lo spazio occupato dal mazzo e dalla briscola a sinistra.

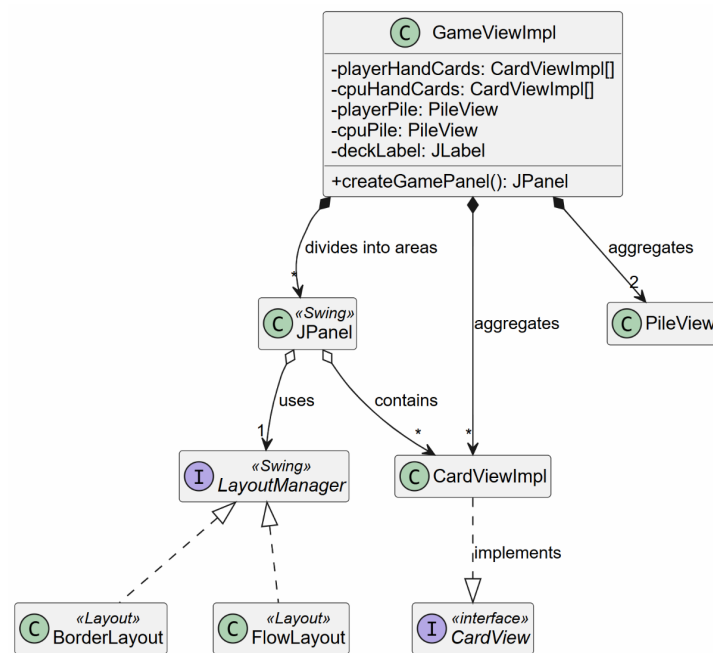


Figura 2.5: Rappresentazione della gestione dei Layout del tavolo da gioco

Rendering delle carte

Problema Per garantire un'esperienza ottimale al giocatore, le carte da gioco devono risultare nitide e ben leggibili. Tuttavia, quando Java importa le immagini originali in formato .png negli spazi dell'interfaccia, utilizza di default un algoritmo di ridimensionamento molto basilare e questo causa un risultato visivo pessimo. Il problema architetturale consiste quindi nel trovare una soluzione che assicuri una resa grafica ottimale, ma senza inserire tutta la logica di rendering nella classe principale `GameViewImpl`.

Soluzione Il problema è stato risolto assegnando la funzione di rendering delle carte all'interno della classe `CardViewImpl`. Invece di delegare il rendering al framework Swing e caricare direttamente l'immagine, il metodo `renderCard(seed, value)` estrae la matrice originale dei pixel del file .png e applica l'algoritmo `Image.SCALE_SMOOTH`.

Questo algoritmo smussa i pixel ricalcolando le transizioni di colore. In questo modo, qualunque tipo di display abbia l'utente, la carta manterrà un'ottima definizione e contorni lisci, garantendo una resa grafica ottima.

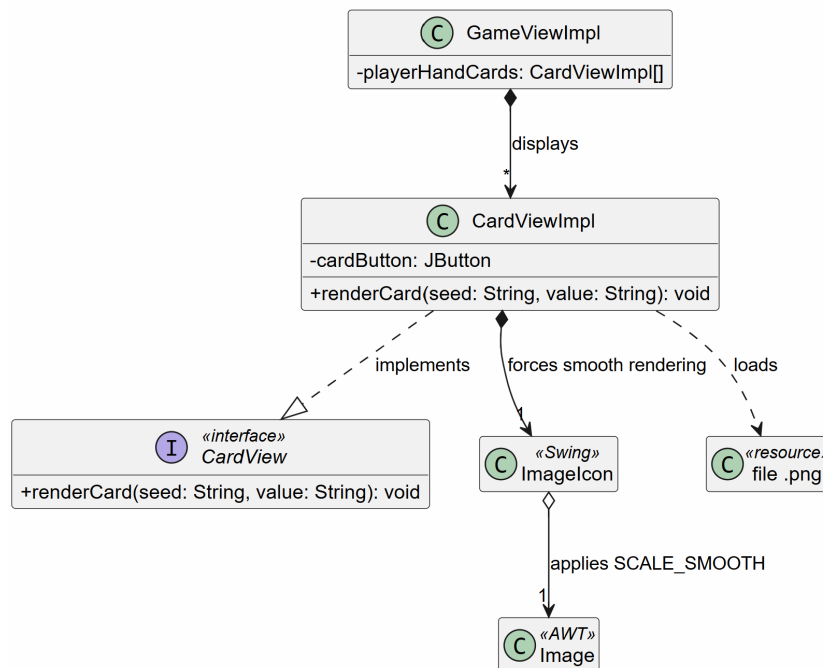


Figura 2.6: Rappresentazione del rendering ottimizzato assegnato a `CardViewImpl`

2.2.3 Maisam Noumi

Incapsulamento del dominio logico

Problema Il sistema di gioco è composto da più componenti che interagiscono tra di loro: il mazzo (**Deck**), i giocatori (**Player**) e il gestore del turno (**RoundManager**). Permettere al Controller di accedere e manipolare direttamente queste classi avrebbe violato il principio di incapsulamento, rischiando di portare il gioco in uno stato inconsistente (ad esempio estraendo una carta dal mazzo fuori dal corretto ordine dei turni).

Soluzione Per nascondere questa complessità, l'interfaccia **GameModel** è stata progettata seguendo il pattern **Facade**. Il Controller non ha alcuna visibilità sui sottomoduli interni, ma interagisce esclusivamente tramite operazioni ad alto livello come **startMatch()** e **endRound()**. Nel corso dello sviluppo è stato inoltre eseguito un refactoring per rendere privati i metodi **assignBriscola()** e **dealInitialCards()**, che erano inizialmente esposti nell'interfaccia pubblica pur essendo dettagli puramente interni alla fase di inizializzazione. In questo modo nessun componente esterno può interferire con lo stato interno della partita.

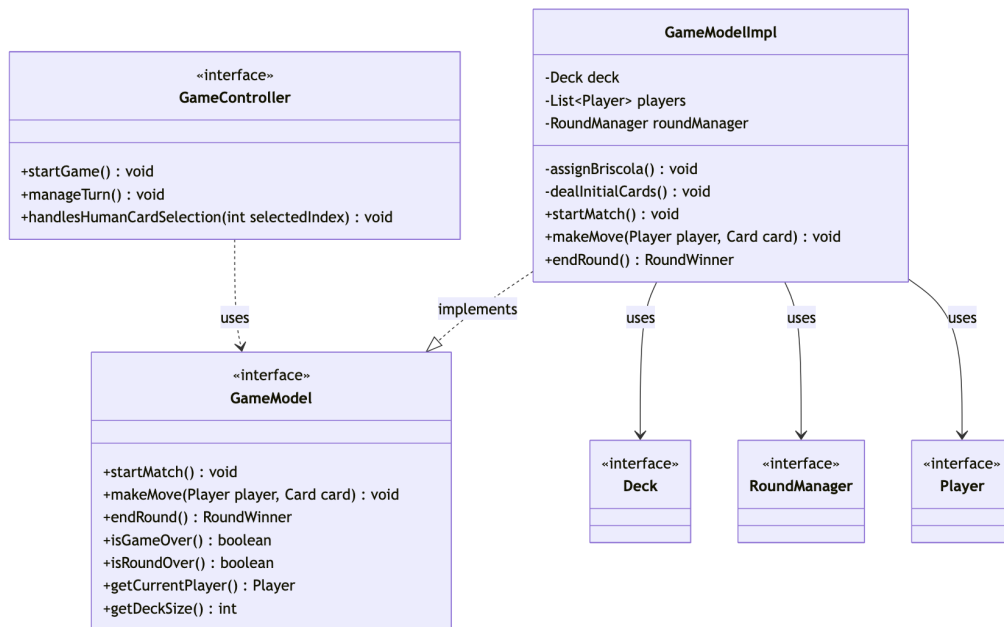


Figura 2.7: Rappresentazione del pattern Facade applicato a **GameModel**.

Reattività dell'interfaccia grafica e gestione asincrona dei turni

Problema Durante il flusso dei turni, il gioco necessita di introdurre delle pause, per simulare il tempo di "riflessione" della CPU e per permettere all'utente di visualizzare l'esito della presa prima che le carte vengano rimosse dal tavolo.

Soluzione Inizialmente, il Controller eseguiva le operazioni in modo sincro-
no, rendendo il gioco visivamente incomprensibile poiché le carte della CPU apparivano e sparivano istantaneamente. Invocare `Thread.sleep()` direttamente sull'Event Dispatch Thread (EDT) per rallentare il flusso avrebbe risolto la tempistica, ma avrebbe causato il congelamento dell'intera interfaccia grafica, impedendo persino il rendering delle carte appena giocate. La soluzione adottata consiste nel delegare queste attese a thread di background separati tramite `new Thread(...).start()`, lasciando l'EDT libero e responsivo. Al termine della pausa, le istruzioni di aggiornamento del Model e della View vengono re-scheduled sull'EDT tramite `SwingUtilities.invokeLater()`, garantendo quindi una corretta sincronizzazione grafica.

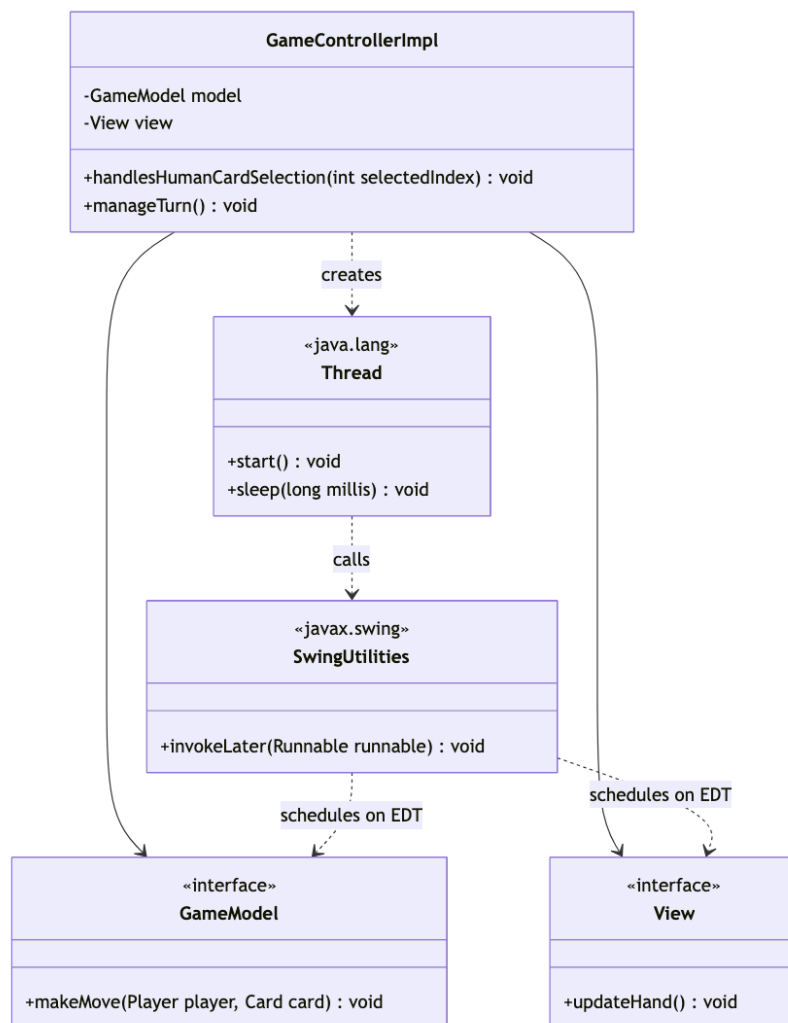


Figura 2.8: Rappresentazione della gestione asincrona dei turni.

Protezione dello stato di gioco da interazioni intempestive

Problema Poiché l'interfaccia grafica rimane responsiva durante le pause asincrone, l'utente ha la possibilità di cliccare sulle proprie carte anche in momenti non opportuni (ad esempio, durante il turno della CPU). Senza alcun controllo preventivo, l'inoltro di questi input al Controller scatenava la seguente eccezione a runtime:

```
Exception in thread "AWT-EventQueue-0"
java.lang.IndexOutOfBoundsException: Index: 2 Size: 2
    at java.base/java.util.ImmutableCollections$List12.get
```

```

at it.unibo.briscoola.model.impl.game.RoundManagerImpl
    .getCurrentPlayer
at it.unibo.briscoola.controller.impl.GameControllerImpl
    .handlesHumanCardSelection

```

La causa risiede nel fatto che `RoundManagerImpl` traccia il turno tramite un indice interno (`currentPlayerIndex`), incrementato a ogni giocata. A round terminato, tale indice assume il valore 2 (in una partita a due giocatori); un ulteriore click intempestivo dell'utente generava una chiamata a `getCurrentPlayer()` che tentava di accedere all'indice 2 di una lista di dimensione 2, causando la rottura del sistema.

Soluzione Per ovviare a questa vulnerabilità, all'inizio del metodo `handlesHumanCardSelection()` sono stati inseriti due controlli sul `Model`. Il primo verifica che la partita e il round siano effettivamente in corso e il secondo si assicura che sia il turno del giocatore umano. Se anche una sola preconditione fallisce, il metodo ritorna immediatamente senza eseguire alcuna operazione sul `Model`. In questo modo, lo stato del gioco è dettato in via esclusiva dal `Model`, e qualsiasi click ripetuto o fuori tempo generato dalla `View` viene scartato in totale sicurezza.

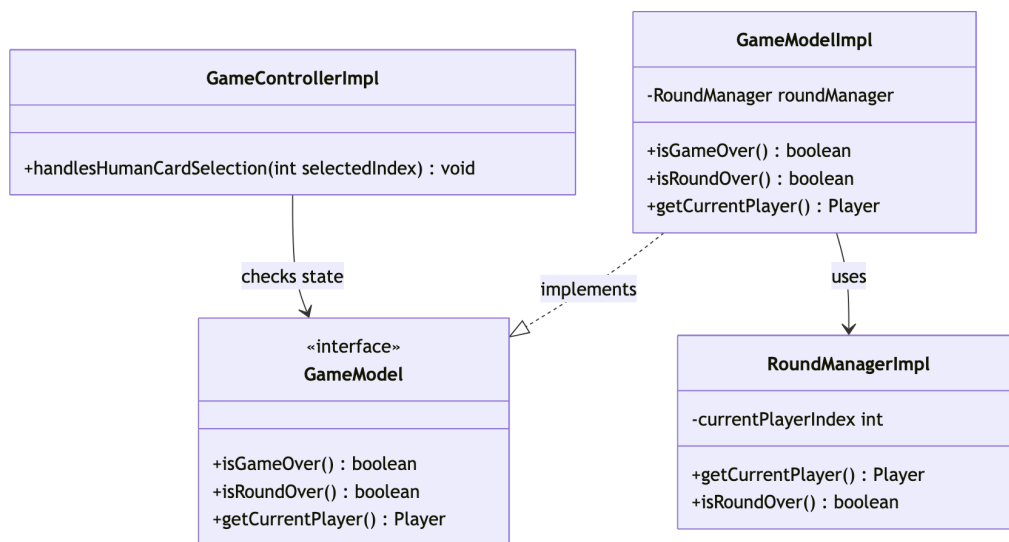


Figura 2.9: Rappresentazione del meccanismo di protezione da interazioni intempestive.

2.2.4 Adam Paolo Razzino

Gestione Cpu e diverse difficoltà

Problema In un gioco a singolo giocatore come briscola è necessario avere un avversario adatto a ogni livello di conoscenza del gioco

Soluzione Per mantenere una struttura scalabile e con una gerarchia immutabile, viene utilizzata un'implementazione di CPU come estensione del giocatore, mantenendo così il polimorfismo fornito dall'interfaccia `Player`.

La classe `CpuPlayer` contiene al suo interno un valore di `PlayStrategy`, il quale caratterizza il livello di difficoltà, ognuno con una diversa implementazione del metodo `cardIndex()`. Il `CpuPlayer` prende come argomento nel costruttore un elemento di un enum `Difficulty`; questo viene passato alla funzione `create` di `StrategyFactory`, la quale provvede a fornire la strategia.

L'utilizzo di un Strategy Pattern permette di creare un ambiente estensibile in caso di implementazione di differenti Strategie e indipendenti dalla struttura del progetto

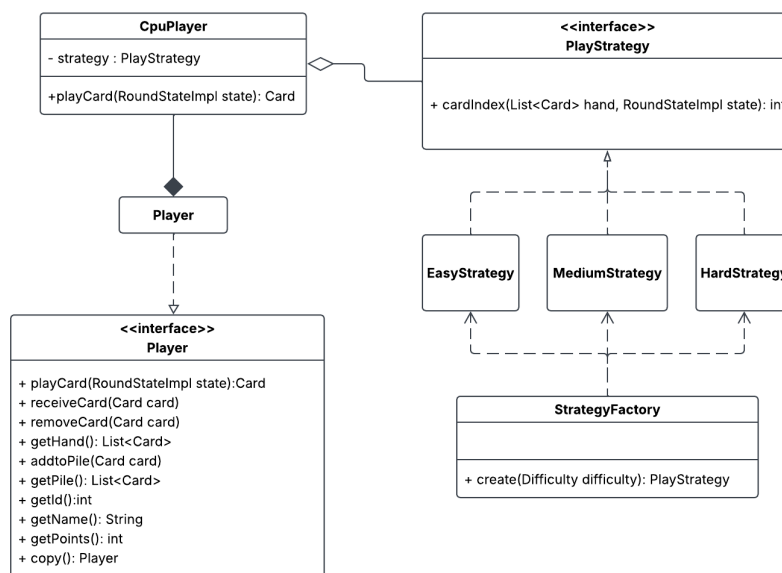


Figura 2.10: Rappresentazione del Plyer CPu e del Pattern Strategy

Gestione della persistenza della leaderboard

Problema Al termine di una partita a punti, per seguire una struttura di tipo *Arcade*; è necessario un processo di salvataggio del punteggio di gioco e del nickname del giocatore

Soluzione È stata implementata l'interfaccia **Leaderboard** che si occupa della memorizzazione temporanea dei punteggi caricati da file. Delega poi il salvataggio permanente su file a una classe che implementa l'interfaccia **ScoreFileManager**. Quest'ultima viene inizializzata con un parametro **fileName** che contiene il nome del file con estensione *json*.

Questo permette alle classi che implementano **MenuController** e **GameController** di utilizzare un'istanza di **Leaderboard** senza essere a conoscenza dei processi I/O.

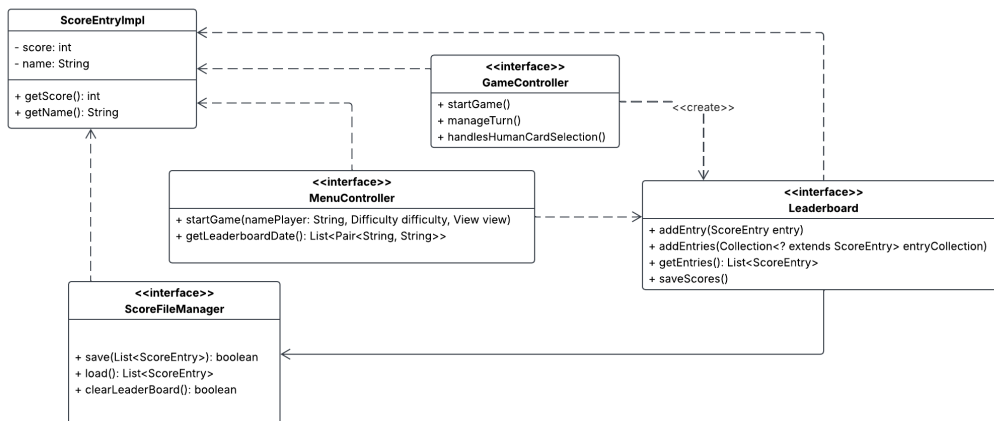


Figura 2.11: Rappresentazione della gestione del salvataggio dati

Gestione dei turni e Classi helper

Problema I turni sono elementi modulari del sistema di gioco, durante i quali le informazioni cambiano continuamente. È necessario inoltre avere una implementazione in grado di permettere a giocatori non umani di comprendere la situazione di gioco.

Soluzione È stata implementata una interfaccia **RoundManager** che si occupa del mantenimento delle informazioni di gioco. Queste informazioni sono poi modulate per creare istanze di **RoundStateImpl**.

RoundStateImpl è un record che mantiene all'interno una copia della lista **RoundPlay** per mantenere la protezione dei dati delle classi. Questo è ciò che permette alle classi implementative di **PlayStrategy** di effettuare le opportune decisioni.

Il **RoundManagerImpl** ha anche un metodo per fornire il vincitore del turno attuale, dopo il quale il tavolo viene svuotato in attesa della nuova lista di giocatori con l'ordine appropriato.

Questa implementazione consente di avere uno scheletro del round altamente scalabile, anche in caso di un possibile aumento di giocatori. Grazie alla mancanza di valori fissi e costrittivi.

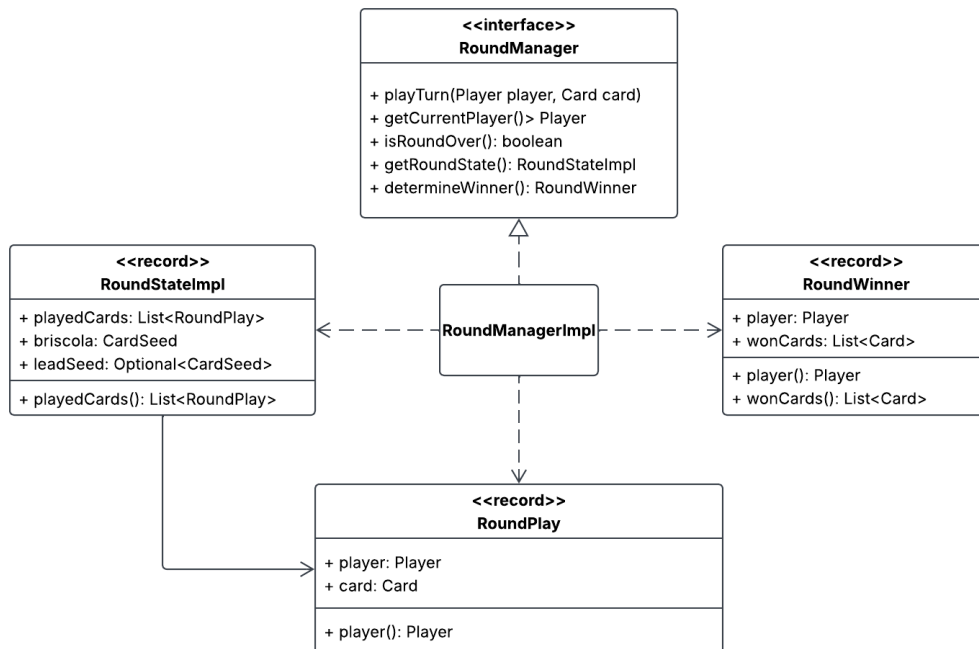


Figura 2.12: Rappresentazione della classe **RoundManagerImpl** e helper

Presentazione a schermo di informazioni tramite Popup

Problema Necessaria la presenza di elementi a schermo per la comunicazione all'utente di diversi messaggi e/o risultati riguardante la partita

Soluzione Implementazione di una factory di **Popup** che propone il metodo **create**, il quale prende in ingresso un elemento enum di tipo **Popups** riguardante il tipo necessitato e un messaggio stringa che verrà mostrato a schermo.

Alla creazione della Factory è necessario fornire un **JRootPane** che verrà utilizzato come schermata genitrice e un **Supplier** che dovrà fornire una lista che verrà utilizzata come dati visualizzati nella leaderboard.

È stata una decisione implementativa l'impossibilità di aprire più di un **Popup** in un momento, questo per non appesantire troppo l'interfaccia grafica

Implementa una struttura con un Pattern Factory per permettere una futura estensione del progetto con la presenza di più **Popup**

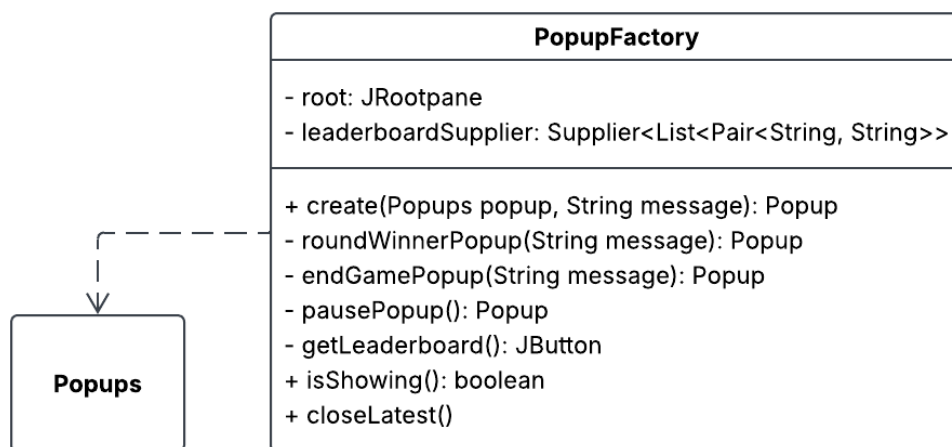


Figura 2.13: Rappresentazione della **PopupFactory** e Enum **Popups**

Capitolo 3

Sviluppo

3.1 Testing automatizzato

In questo progetto abbiamo implementato dei test unitari con JUnit 5 per valutare il corretto eseguimento di tutte le classi principali. Riportiamo di seguito le descrizioni dei test presenti:

- **StandardCardImplTest**: testa il comportamento di una carta standard controllando che carte differenti vengano riconosciute effettivamente come differenti.
- **MenuControllerTest**: testa il comportamento del `MenuController` in caso di situazioni anomale, come l'inserimento di difficoltà inconsistenti o nomi di `Player` non accettabili.
- **DeckImplTest**: testa il corretto funzionamento del deck, la memorizzazione delle carte controllando l'assenza di carte doppioni nel mazzo, verificando che il mazzo risulti effettivamente vuoto al termine dell'estrazione delle carte e che la carta mostrata di Briscola sia effettivamente l'ultima carta ad essere pescata dai giocatori.
- **RoundManagerTest**: testa la correttezza dello stato di gioco e dello svolgimento di un singolo turno, compresa la decisione del vincitore.
- **LeaderboardTest**: testa la memorizzazione dei dati e il corretto ordinamento di essi
- **ScoreFileManagerTest**: testa processi di I/O tramite un file di testing

- **GameModelTest**: verifica l'intero ciclo di vita del gioco, dalla corretta inizializzazione della partita fino alla risoluzione completa del round.
- **CpuPlayerTest**: verifica il corretto comportamento dei giocatori Cpu in diverse situazioni. Testa inoltre il corretto svolgimento delle differenti difficoltà

3.2 Note di sviluppo

3.2.1 Adam Paolo Razzino

Utilizzo di Stream

Utilizzati frequentemente, specialmente per il confronto delle carte in diverse situazioni. Permalink di un esempio:

```
https://github.com/rAdamm0/00P25-briscoola/blob/0822df5b90d353f7b5e8d988f05bfafcefc59d9d/src/main/java/it/unibo/briscoola/model/impl/game/RoundManagerImpl.java#L102
```

Utilizzo di Gson

Utilizzato per semplificare il processo di salvataggio di informazioni su file. Permalink di un esempio:

```
https://github.com/rAdamm0/00P25-briscoola/blob/0822df5b90d353f7b5e8d988f05bfafcefc59d9d/src/main/java/it/unibo/briscoola/model/impl/leaderboard/ScoreFileManagerImpl.java#L106
```

Utilizzo di Optional

Utilizzati in casi in cui il valore potrebbe essere assente. Permalink di esempio:

```
https://github.com/rAdamm0/00P25-briscoola/blob/0822df5b90d353f7b5e8d988f05bfafcefc59d9d/src/main/java/it/unibo/briscoola/model/impl/player/cpu/strategies/MediumStrategy.java#L18-L19
```

Utilizzo di SLF4J

Utilizzato in caso di errori recuperabili tramite `try...catch`. Permalink di esempio:

<https://github.com/rAdamm0/00P25-briscoola/blob/0822df5b90d353f7b5e8d988f05bfafcefc59d9d/src/main/java/it/unibo/briscoola/model/impl/leaderboard/ScoreFileManagerImpl.java#L110>

Utilizzo di generici

Utilizzati in una classe Pair, il cui codice preso da un esercizio di laboratorio

<https://github.com/rAdamm0/00P25-briscoola/blob/2eceb5e8887277f82f7b5b29a4209d6b7cfd88d0/src/main/java/it/unibo/briscoola/controller/impl/utils/Pair.java>

3.2.2 Andrea Reggiani

Utilizzo di Generics

Utilizzati per la gestione delle Carte all'interno di un mazzo. Permalink di un esempio:

<https://github.com/rAdamm0/00P25-briscoola/blob/47e416a81bfe82bc36c1f8891f203ef604526964/src/main/java/it/unibo/briscoola/model/impl/deck/AbstractDeckImpl.java#L19>

Utilizzo di Lambdas

Utilizzati per implementare il disaccoppiamento delle responsabilità:

<https://github.com/rAdamm0/00P25-briscoola/blob/47e416a81bfe82bc36c1f8891f203ef604526964/src/main/java/it/unibo/briscoola/view/impl/GridViewImpl.java#L120>

Utilizzo di GridBagConstraints

Utilizzati per gestire layout manager i menu iniziali:

<https://github.com/rAdamm0/00P25-briscoola/blob/47e416a81bfe82bc36c1f8891f203ef604526964/src/main/java/it/unibo/briscoola/view/impl/menu/MainMenu.java#L46>

Utilizzo di ActionListener

Utilizzati per gestire l'input da parte del giocatore nei i menu iniziali di selezione della modalità di gioco:

<https://github.com/rAdamm0/00P25-briscoola/blob/2eceb5e8887277f82f7b5b29a4209d6b7cfd88d0/src/main/java/it/unibo/briscoola/view/impl/menu/MainMenu.java#L60>

3.2.3 Riem Boukhama

Utilizzo di Pair

Utilizzato per ricevere una lista di coppie seme, valore per aggiornare la mano del giocatore:

```
https://github.com/rAdamm0/00P25-briscoola/blob/2eceb5e8887277f82f7b5b29a4209d6b7cfd88d0/src/main/java/it/unibo/briscoola/view/impl/GameViewImpl.java#L316
```

Utilizzo di Image.SCALE_SMOOTH

Utilizzato per ridimensionare le carte e renderizzarle con una buona resa grafica:

```
https://github.com/rAdamm0/00P25-briscoola/blob/2eceb5e8887277f82f7b5b29a4209d6b7cfd88d0/src/main/java/it/unibo/briscoola/view/impl/CardViewImpl.java#L89
```

Utilizzo del metodo getResource() della classe ClassLoader

Utilizzato per caricare i png delle carte senza dipendere dal percorso assoluto:

```
https://github.com/rAdamm0/00P25-briscoola/blob/2eceb5e8887277f82f7b5b29a4209d6b7cfd88d0/src/main/java/it/unibo/briscoola/view/impl/CardViewImpl.java#L85
```

3.2.4 Maisam Noumi

Gestione della concorrenza: Thread

Utilizzati per non bloccare l'interfaccia grafica durante le pause di gioco:

```
https://github.com/rAdamm0/00P25-briscoola/blob/2eceb5e8887277f82f7b5b29a4209d6b7cfd88d0/src/main/java/it/unibo/briscoola/controller/impl/GameControllerImpl.java#L108
```

Utilizzo di Optional

Utilizzato ad esempio all'interno dell'interfaccia di `GameModelImpl` per gestire l'eventualità in cui il mazzo sia vuoto o la briscola non sia ancora stata assegnata.

```
https://github.com/rAdamm0/00P25-briscoola/blob/2eceb5e8887277f82f7b5b29a4209d6b7cfd88d0/src/main/java/it/unibo/briscoola/model/impl/game/GameModelImpl.java#L86
```

Utilizzo di Stream API e Lambda Expressions

Utilizzati ad esempio, all'interno di `GameModelImpl.endRound()`, per ritrovare l'istanza corretta del giocatore vincitore confrontandone l'ID:

<https://github.com/rAdamm0/00P25-briscoola/blob/2eceb5e8887277f82f7b5b29a4209d6b7cfd88d0/src/main/java/it/unibo/briscoola/model/impl/game/GameModelImpl.java#L158>

Utilizzo di Record Types

Usato il record `RoundWinner` per incapsulare in modo immutabile l'esito di una mano:

<https://github.com/rAdamm0/00P25-briscoola/blob/2eceb5e8887277f82f7b5b29a4209d6b7cfd88d0/src/main/java/it/unibo/briscoola/controller/impl/GameControllerImpl.java#L109>

Capitolo 4

Commenti finali

In quest'ultimo capitolo si tirano le somme del lavoro svolto e si delineano eventuali sviluppi futuri.

Nessuna delle informazioni incluse in questo capitolo verrà utilizzata per formulare la valutazione finale, a meno che non sia assente o manchino delle sezioni obbligatorie. Al fine di evitare pregiudizi involontari, l'intero capitolo verrà letto dai docenti solo dopo aver formulato la valutazione.

4.1 Autovalutazione e lavori futuri

4.1.1 Adam Paolo Razzino

Per quanto riguarda l'autovalutazione sono piuttosto soddisfatto del lavoro svolto, ma so che avrei potuto fare di meglio. È sicuramente stata un'esperienza formativa piuttosto importante per il progresso personale, soprattutto in area di progettista, ma mi rendo conto di non aver posto attenzione allo sviluppo di questo progetto con adeguata costanza e tempo. Ho imparato che di fronte a un progetto di questo tipo la parte fondamentale per il suo svolgimento è la capacità di gestire il tempo in maniera adeguata, soprattutto considerando la preparazione di diversi esami in parallelo. Ho intenzione di mantenere questo progetto e ampliare il mio repertorio implementando un'espansione mantenendo l'architettura Model-View-Controller (MVC). Considerando come il Model sia già in grado di gestire un tavolo con più giocatori è necessario uno sviluppo ulteriore della view, aggiungendo elementi modulari in grado di poter visualizzare la presenza di ulteriori giocatori senza cambiare l'essenza del gioco.

4.1.2 Andrea Reggiani

A termine progettazione mi ritengo generalmente soddisfatto di questo progetto e di come la coesione del gruppo si sia consolidata in maniera positiva nel corso di sviluppo.

Il nostro gruppo, nonostante gli impegni dovuti a lezioni e altri esami in corso, é riuscito a sviluppare un applicativo funzionale, considerando soprattutto il forte obbligo di rispettare l'architettura MVC, che, sia nei primi periodi, che nelle ultime settimane di sviluppo, è rimasto un vincolo difficile da soddisfare.

Ritengo che sarebbe stato possibile implementare meglio molte delle mie classi, soprattutto le classi di *View*, che sono state buona parte delle mie responsabilità.

Affermo questo tenendo conto che l'implementazione delle classi di *View* hanno subito variegata modifiche nel corso dello sviluppo, in maniera particolare per aderire completamente al pattern MVC.

Avrei preferito concentrarmi su ruoli di sviluppo incentrato maggiormente verso il *Controller* con un maggiore volume di classi. La mia responsabilità si è dovuta indirizzare alla implementazione e gestione della classe *MenuController* e la corrispettiva implementazione *MenuControllerImpl*, sviluppando un singolo sistema che facesse da filtro per le transizioni dal menù principale e l'avvio della partita.

Reputo che la sfida maggiore sia stata mantenere una *Divisione dei ruoli* netta tra *View* e *Controller*.

In previsione di lavori futuri ed espandibilità del software, i prossimi passi si concentrerebbero sul miglioramento delle tecniche di ridimensionamento dinamico della board di gioco e delle carte grafiche, permettendo una maggiore dinamicità delle componenti su schermi differenti.

Complessivamente credo di aver fatto un buon lavoro, nonostante le numerose modifiche al codice, e di essermi coordinato bene con gli altri colleghi.

4.1.3 Riem Boukhama

In conclusione sono generalmente soddisfatto del lavoro svolto, questo progetto mi ha permesso di mettere in concreto concetti visti nel corso e strumenti di programmazione strutturati. Lavorare in team mi ha anche permesso di comprendere le sfide di coordinazione del lavoro da svolgere e le difficoltà di non poter sviluppare codice in totale indipendenza, anche se d'altronde, lavorare in gruppo ha anche permesso di formare un lavoro finale completo e di buon livello. Per quanto riguarda la mia parte individuale, ritengo di aver raggiunto un buon risultato finale, anche se avrei potuto fare mi-

gliamenti sulla dimensione del codice sulla classe di View. Tra le difficoltà riscontrate è stato presente il fatto di rendere rigoroso il pattern architetturale Model-View-Controller, dovendo ricorrere a strutture dati rappresentative per adattare la View al Model e Action Listener che permettessero un collegamento indiretto con il controller. Sono comunque soddisfatto dell'impegno dedicato e tengo in considerazione un completamento del gioco BriscOOla per renderlo ufficiale e completo di feature.

4.1.4 Maisam Noumi

Sviluppare le logiche del Model e del Controller principale è stata una sfida estremamente formativa. La difficoltà maggiore si è presentata nel far comunicare il motore di gioco con l'interfaccia grafica. Inizialmente, la gestione dei turni e delle pause causava blocchi della GUI. Approfondire il funzionamento dell'Event Dispatch Thread di Swing e riprogettare il `GameController` tramite chiamate asincrone ha rappresentato una grande soddisfazione personale. Sono inoltre soddisfatto dell'architettura ottenuta, ovvero un rigoroso disaccoppiamento MVC, che rende il `GameModel` indipendente dalla visualizzazione, rendendo il codice testabile e riutilizzabile. Riguardo al processo di sviluppo, avrei potuto gestire meglio la pianificazione del tempo, bilanciando con maggiore equilibrio il carico di lavoro rispetto agli altri esami. Per quanto concerne gli sviluppi futuri, a livello grafico mi piacerebbe implementare un sistema di animazioni fluide per la transizione delle carte. Questo potrebbe essere realizzato sia arricchendo l'attuale implementazione Swing, sia valutando una migrazione a JavaFX per sfruttarne le librerie di animazioni native.

Appendice A

Guida utente

Durante tutta la durata della schermata di gioco è possibile visualizzare un popup di pausa tramite il tasto **ESC**.

A.1 Menu Principale

All'avvio dell'applicazione viene presentata la schermata principale, composta da tre pulsanti. Il primo avvia la creazione di una nuova partita, il secondo mostra le regole di gioco e il terzo chiude l'applicazione.

Selezionando il primo pulsante, viene richiesto l'inserimento di un username ("Player" di default). Se l'username supera i 15 caratteri, verrà sostituito automaticamente da *DefaultPal*.

Successivamente è necessario selezionare la difficoltà per avviare la partita.

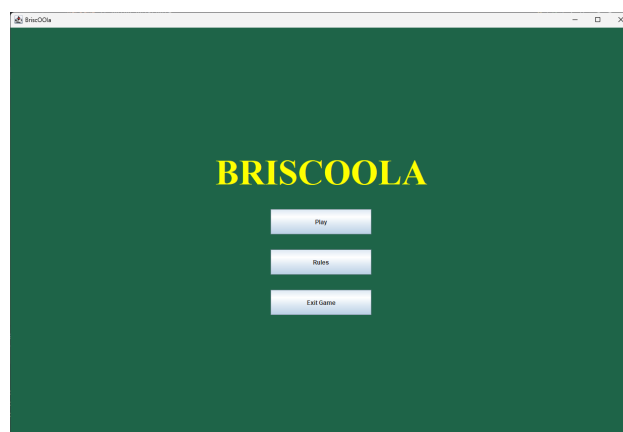


Figura A.1: Schermata del menu principale.

A.2 Fase di Gioco

Durante la partita, il giocatore dispone di un massimo di 3 carte in mano. Sul lato sinistro della schermata si potrà vedere il mazzo e la carta di briscola scoperta. A destra di ciascuna mano è presente la pila delle carte vinte e un contatore del numero di carte prese.

Al termine di ogni turno compare un popup non bloccante che informa il giocatore dell'esito del round.

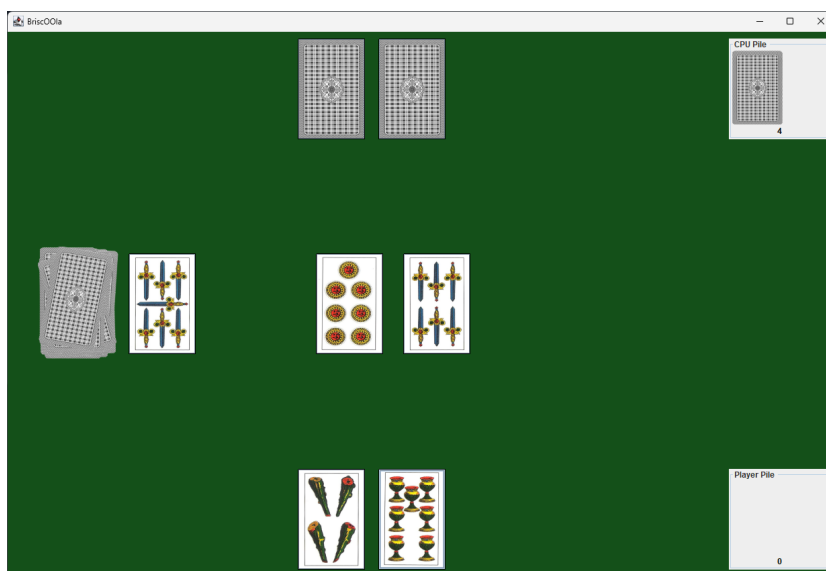


Figura A.2: Fase di gioco

A.3 Termine della partita

Al termine della partita viene visualizzato un popup che mostra il punteggio del giocatore e quello dell'avversario. Questo popup presenta tre pulsanti: il primo riporta alla schermata principale, il secondo apre la leaderboard e il terzo chiude l'applicazione.

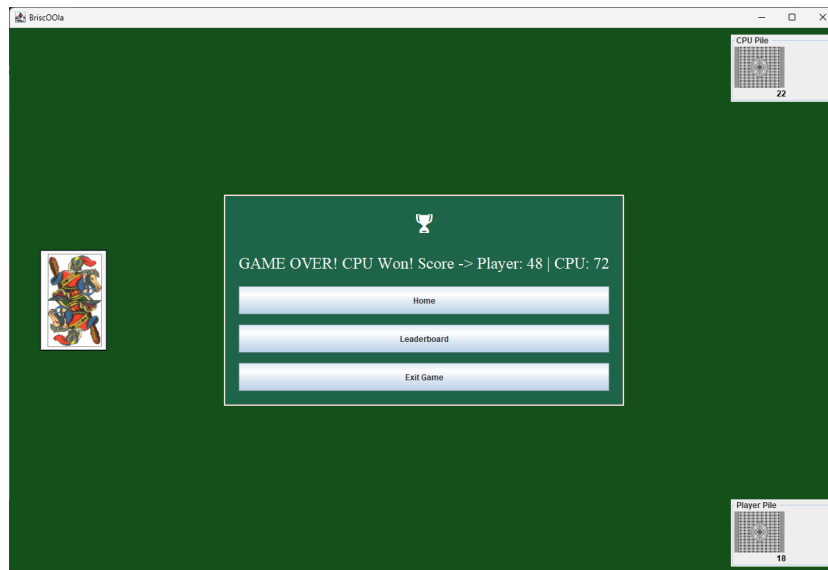


Figura A.3: Popup di fine partita

A.4 Visualizzazione Leaderboard

Il punteggio viene salvato su file esclusivamente in caso di vittoria e viene moltiplicato in base alla difficoltà selezionata. La schermata della leaderboard mostra i dieci punteggi più alti registrati, con il primo classificato evidenziato in cima alla classifica.

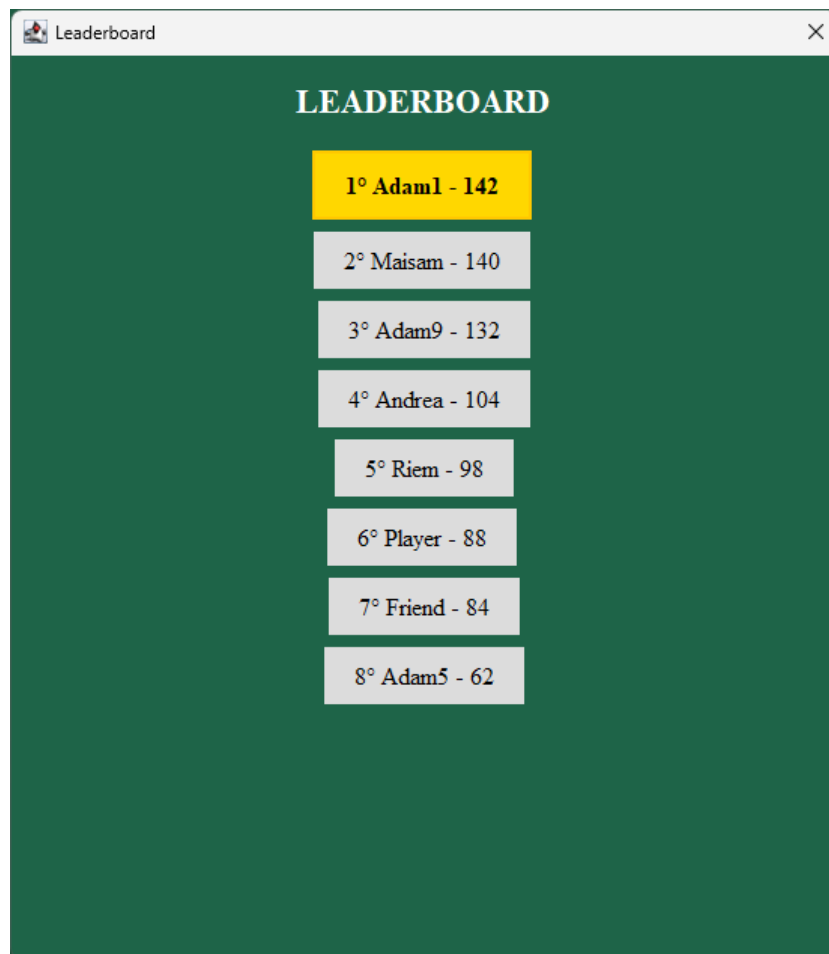


Figura A.4: Schermata della leaderboard.

Appendice B

Esercitazioni di laboratorio

B.1 `adampaolo.razzino@studio.unibo.it`

- Laboratorio 11: <https://virtuale.unibo.it/mod/forum/discuss.php?d=210617#p289560>
- Laboratorio 10: <https://virtuale.unibo.it/mod/forum/discuss.php?d=209589#p288323>
- Laboratorio 09: <https://virtuale.unibo.it/mod/forum/discuss.php?d=208718#p286632>
- Laboratorio 08: <https://virtuale.unibo.it/mod/forum/discuss.php?d=207921#p286070>